Машинное обучение

Лекция 6

Решающие деревья.

Михаил Гущин

mhushchyn@hse.ru

НИУ ВШЭ, 2026

Эволюция решающих деревьев







Решающие деревья



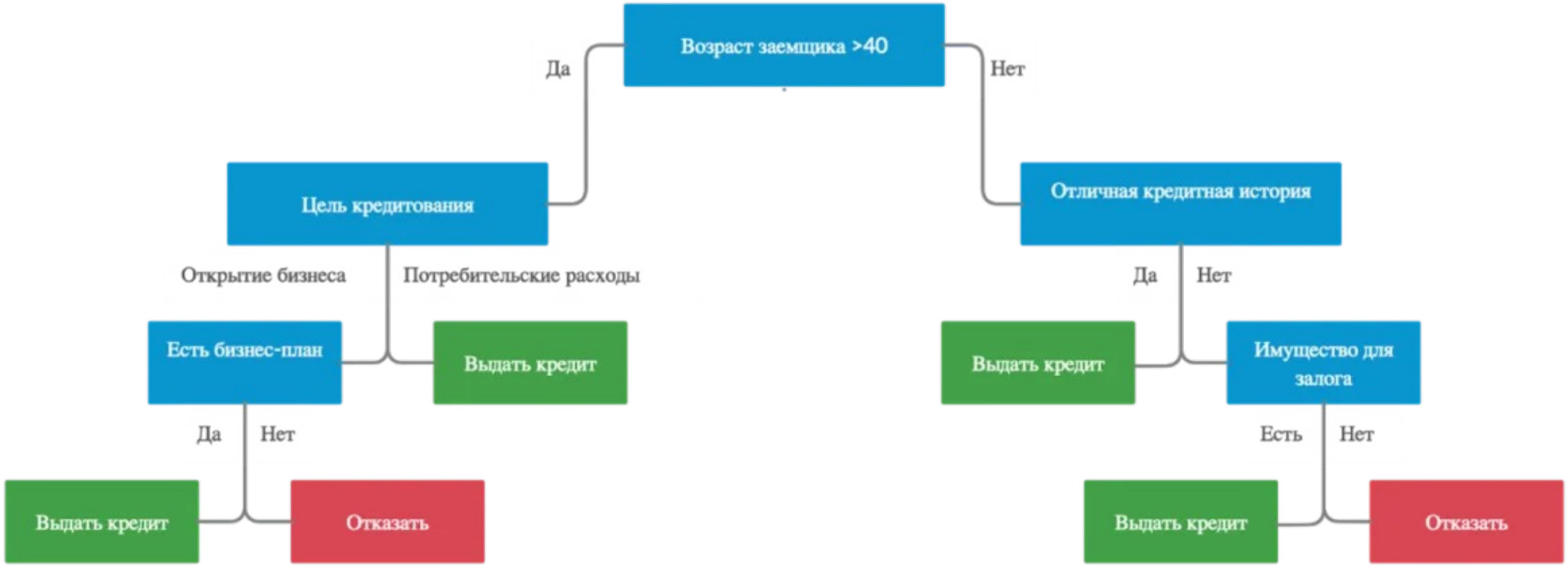




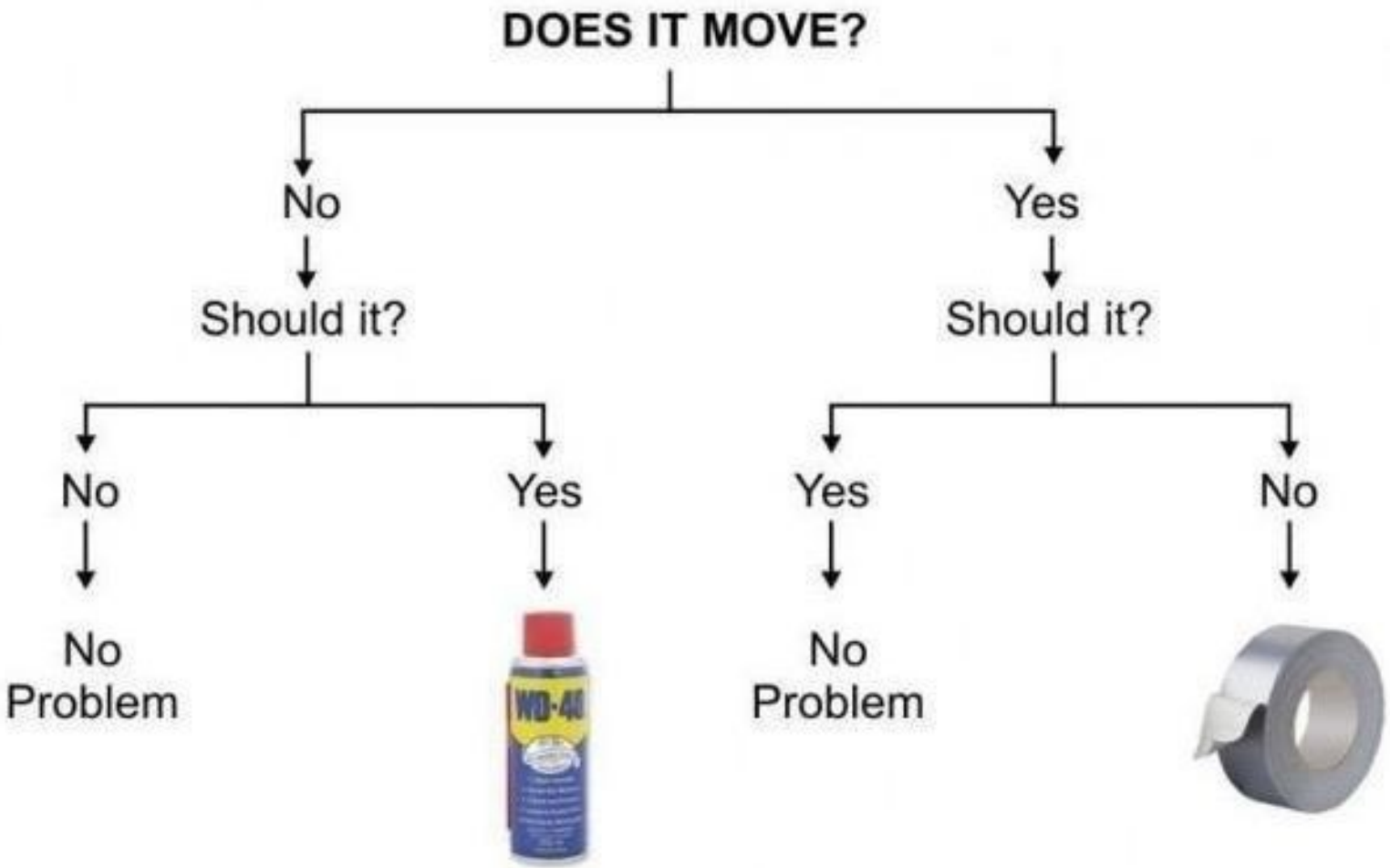
Интуиция



Выдача кредита

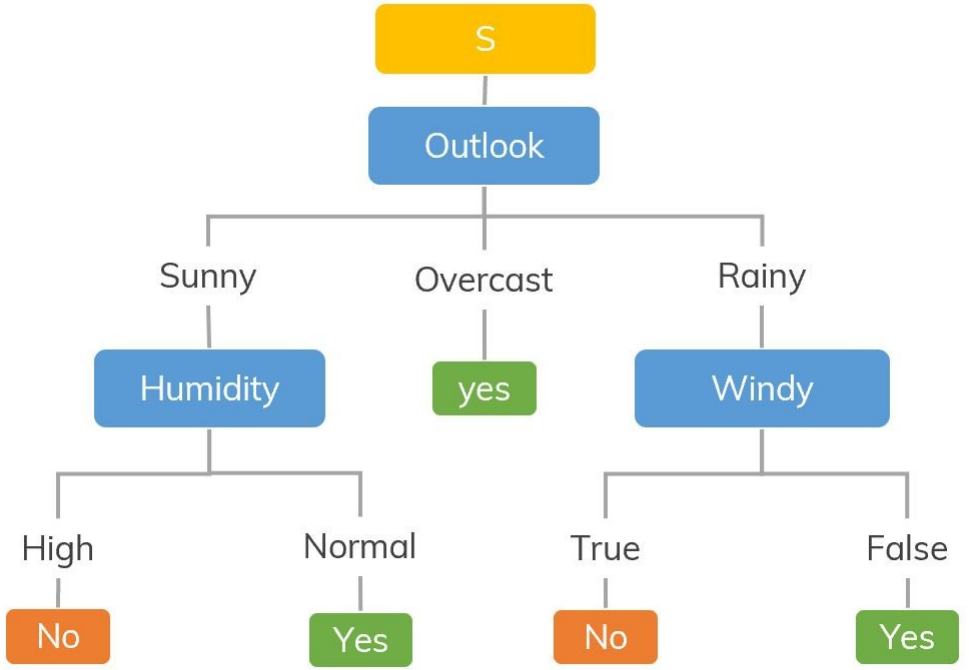


Ремонт


DOES IT MOVE?
No
Yes
Should it?
Should it?
No
Yes
Yes
No
No Problem
No Problem
WD-40

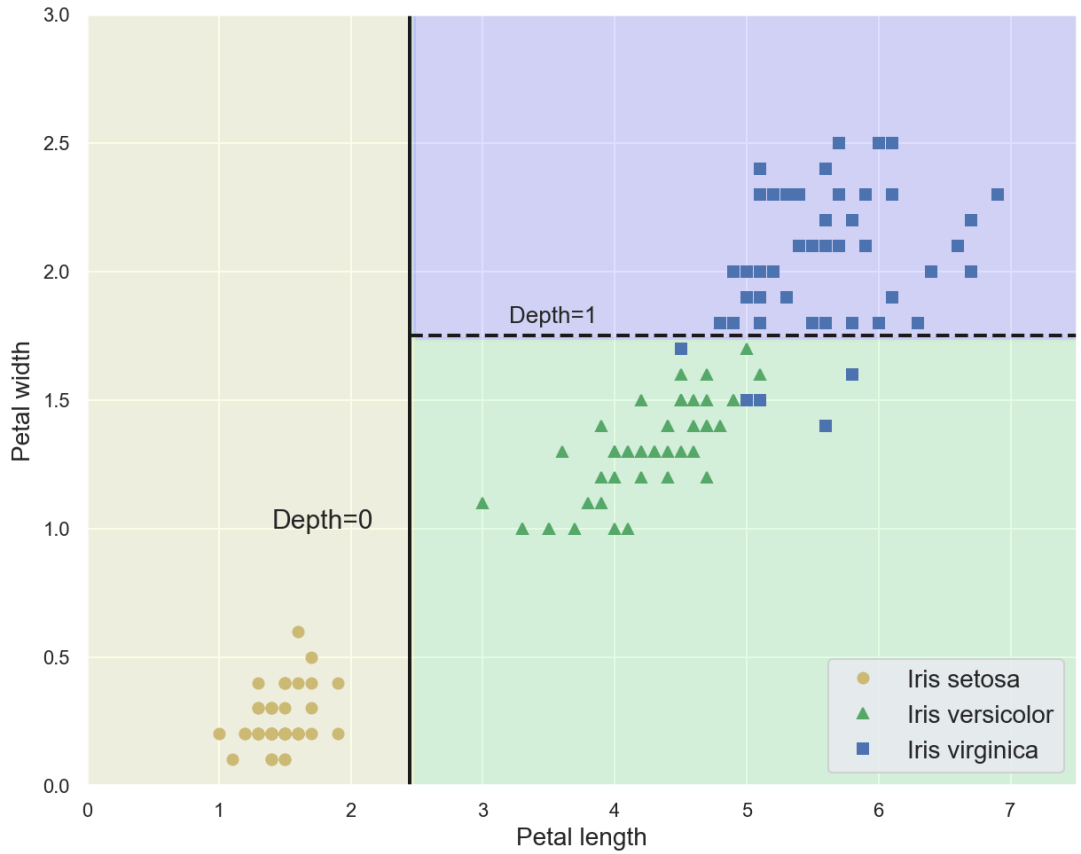
Игра в гольф

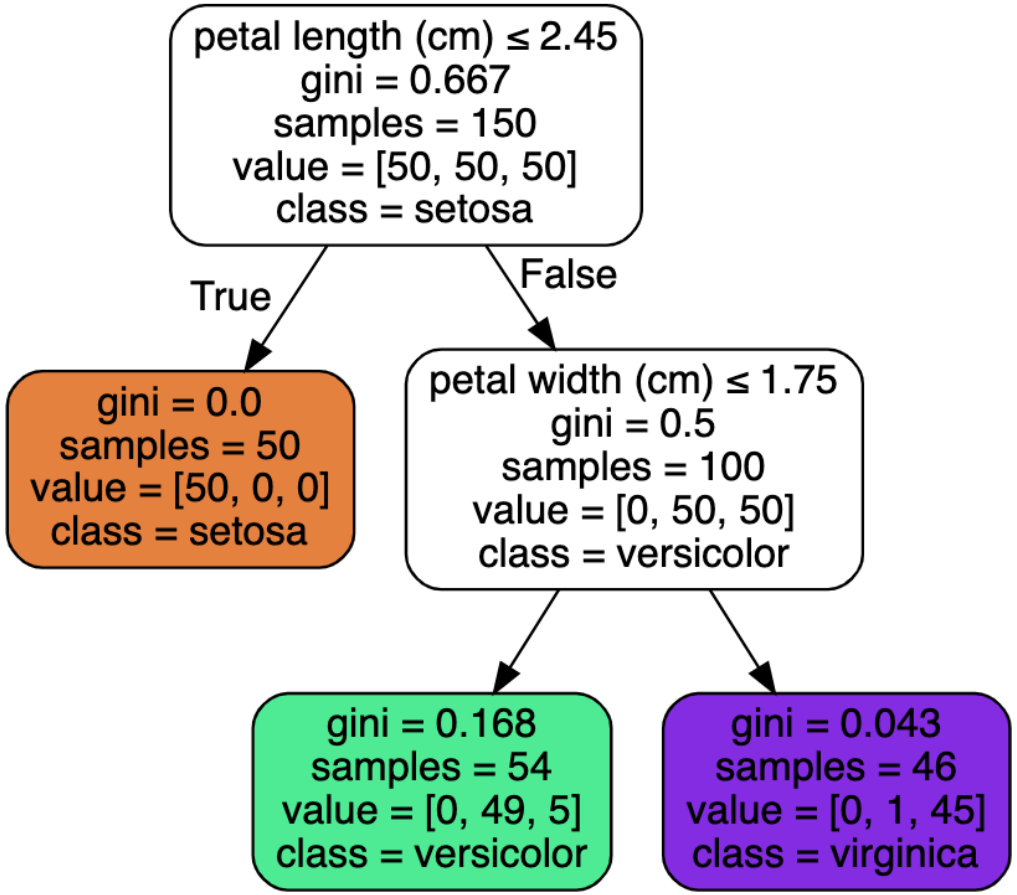
Outlook	Temperature	Humidity	Windy	Play Golf
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No



https://dinhanhthi.com/decision-tree-classifier/

Ирисы ☺

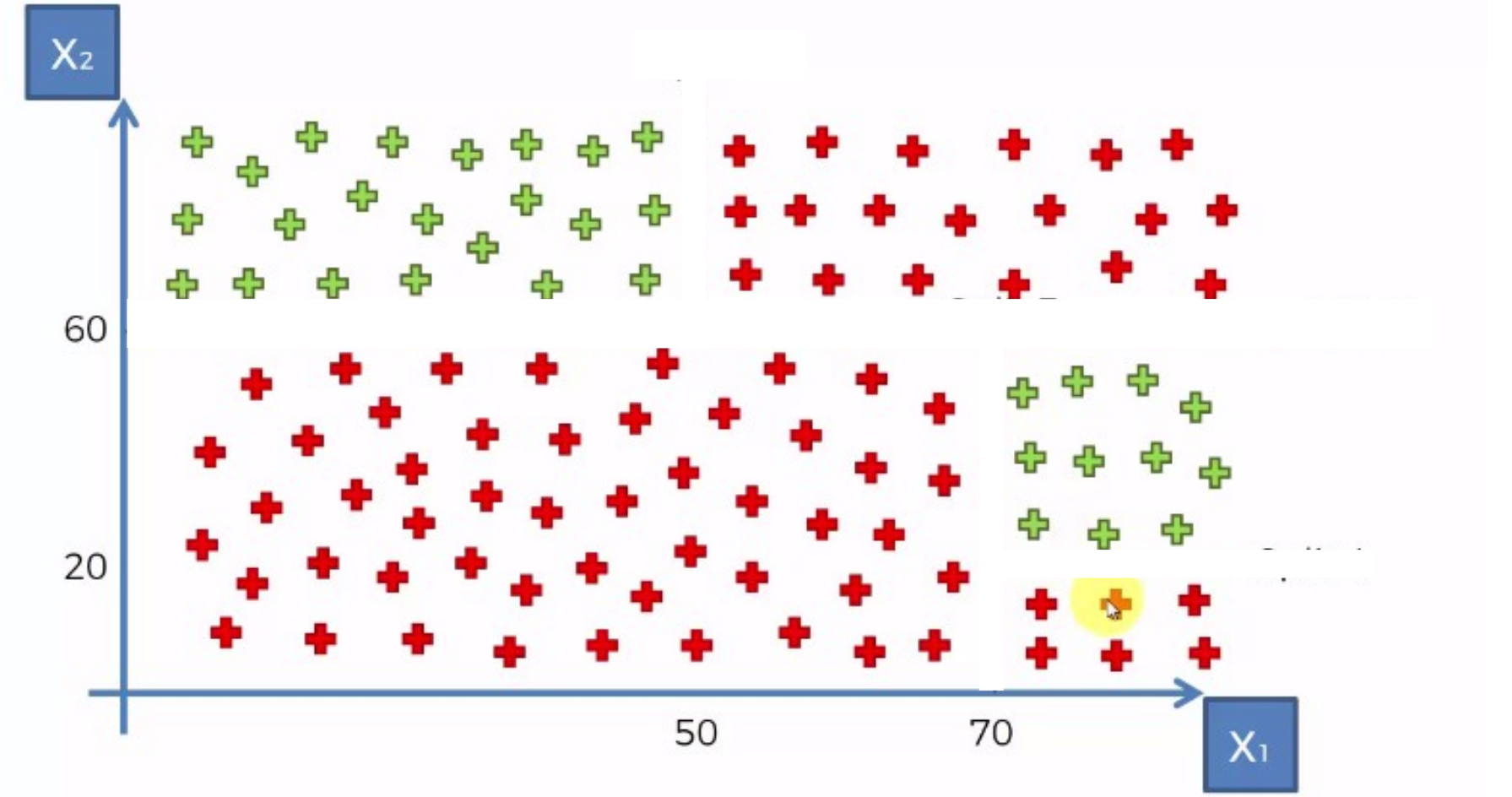

Depth=1
Depth=0
Petal width
Petal length
Iris setosa
Iris versicolor
Iris virginica


petal length (cm) ≤ 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
petal width (cm) ≤ 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica

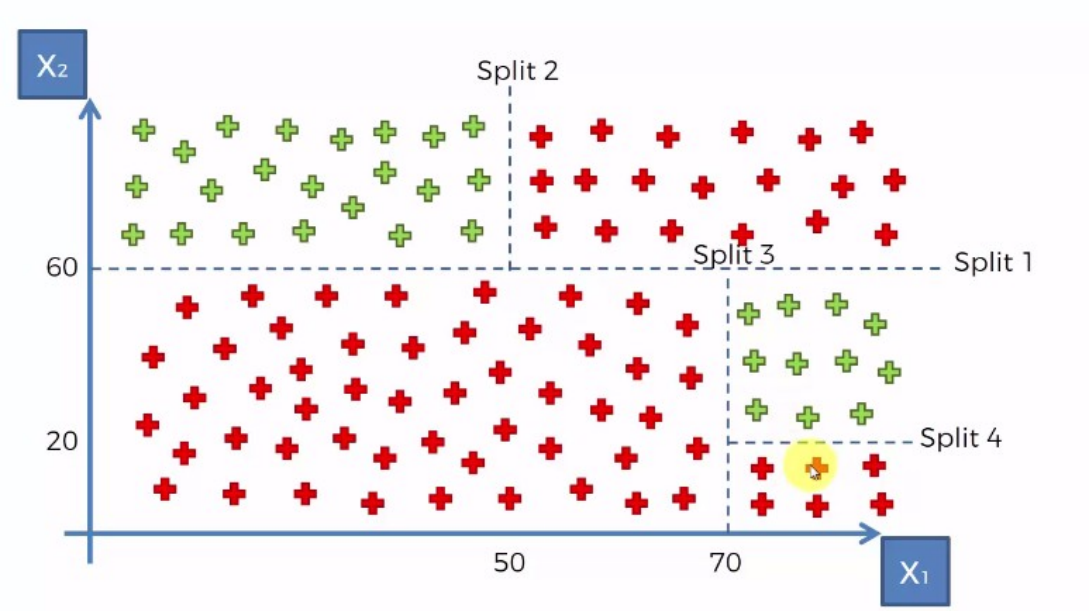
Двоичные деревья

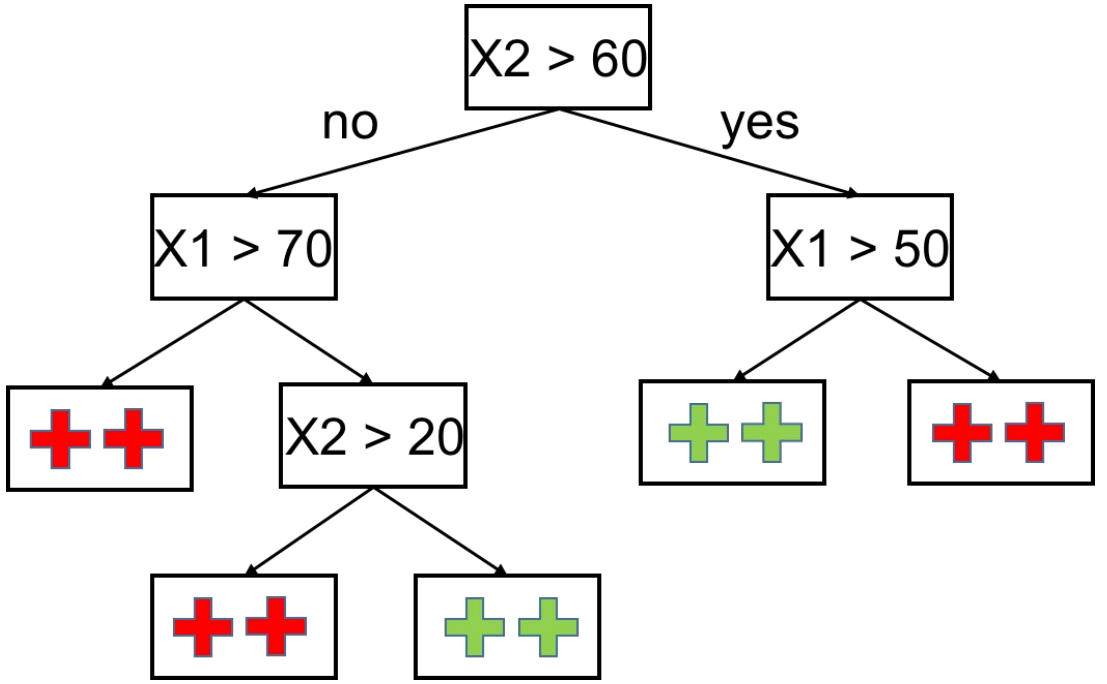


Задача классификации



Решение


X2
Split 2
60
Split 3
Split 1
20
Split 4
50
70
X1


X2 > 60
no
yes
X1 > 70
X1 > 50
X2 > 20

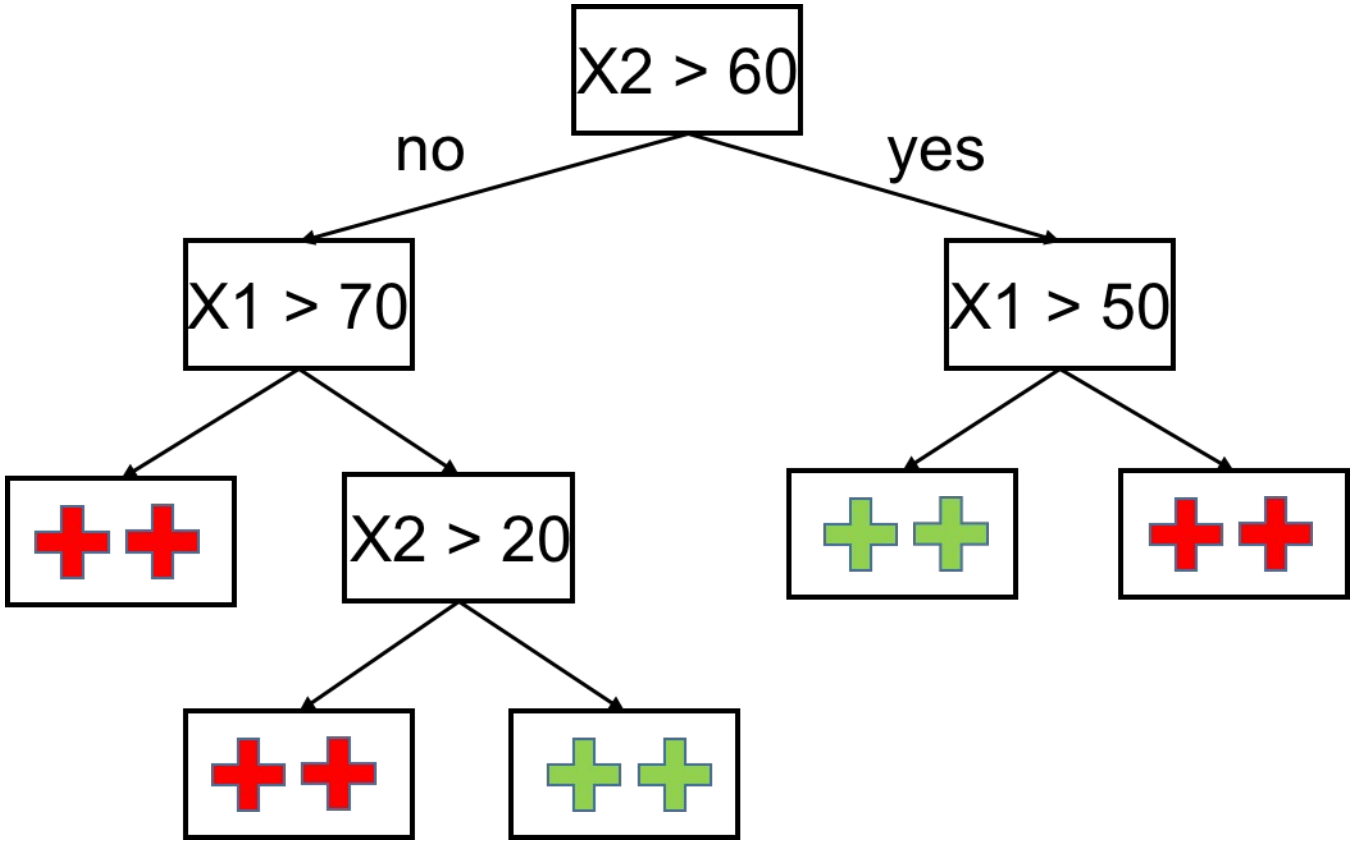
Двоичное решающее дерево

- Дерево начинается из **одной** корневой вершины
- Данные X внутри вершины разбиваются по **одному** входному признаку. Это условие называют **предикатом**: $\{x_j > t\}$
- Каждая вершина дерева имеет **два потомка**: правый и левый

$$R_r(j,t) = \{x | x_j > t\}$$
$$R_l(j,t) = \{x | x_j \leq t\}$$

- Вершины без потомков – **листья** дерева

Двоичное дерево



Рекурсивная функция



Жадный алгоритм построения дерева

```
1. def decision_tree(X, y):

2.     if termination_criterion(X, y) == True:

3.         R = create_leaf_with_prediction(y)

4.     else:

5.         R = create_node()
6.         (X_1, y_1), (X_2, y_2) = best_split(X, y)

7.         for i in [1, 2]:
8.             C = decision_tree(X_i, y_i)
9.             connect_nodes(R, C)

10.    return R
```





Вопросы

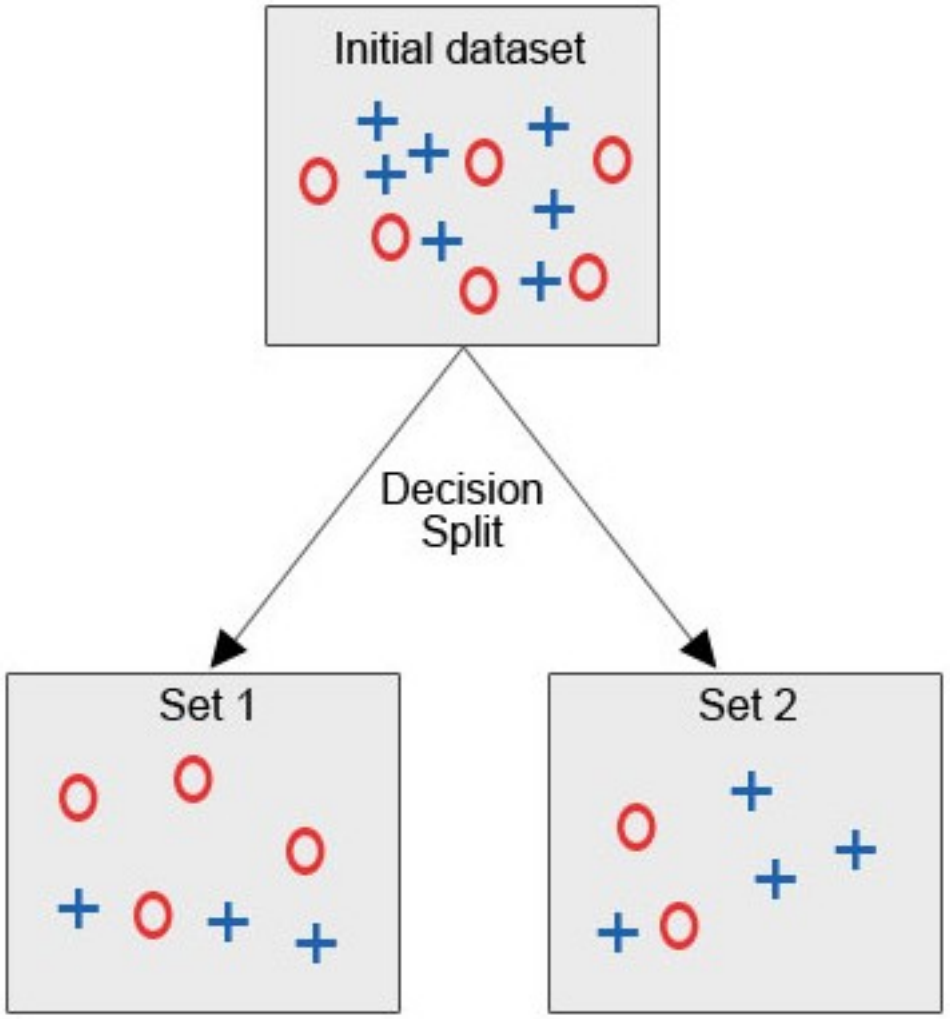
- Как находить предикаты для каждой вершины дерева?
- Какие есть критерии останова?

Критерии информативности для классификации



Выбор предиката

- Предикат делит вершину на две дочерние: левую и правую.
- Как понять, что разделение хорошее?



Функционал качества предиката

- Для поиска лучшего предиката для вершины дерева зададим его функционал качества:

$$\Delta I_{node} = I_{node} - \left(I_{left} \frac{N_{left}}{N_{node}} + I_{right} \frac{N_{right}}{N_{node}} \right) \to \max_{j,t}$$

- I_{node} - **критерий информативности** для вершины
- N_{node} - число объектов в левой вершине дерева
- ΔI_{node} - функционал качества предиката (**information gain**)

Критерий информативности

- Чем **меньше разнообразие** целевой переменной внутри вершины, тем **меньше** должно быть **значение** критерия информативности
- Соответственно, мы максимизируем функционал качества для каждого предиката

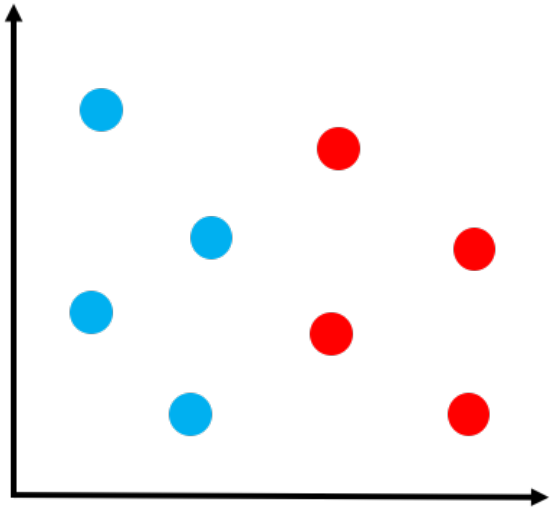
Критерий Джини (Gini)

$$I_{node} = \sum_{k=1}^{K} p_k(1 - p_k)$$

$$p_k = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} [y_i = k]$$

- p_k–вероятность класса для вершины дерева
- K – общее число классов в задаче
- N_{node} – число объектов в вершине дерева

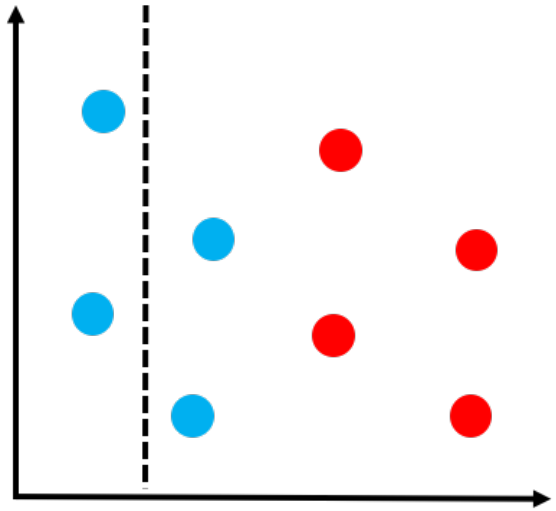
Пример



$p = \frac{4}{8}, p = \frac{4}{8}$

$$I = \frac{4}{8}\left(1 - \frac{4}{8}\right) + \frac{4}{8}\left(1 - \frac{4}{8}\right) = \frac{1}{2}$$

Bad split



$p = \frac{2}{2}, p = \frac{0}{2}$

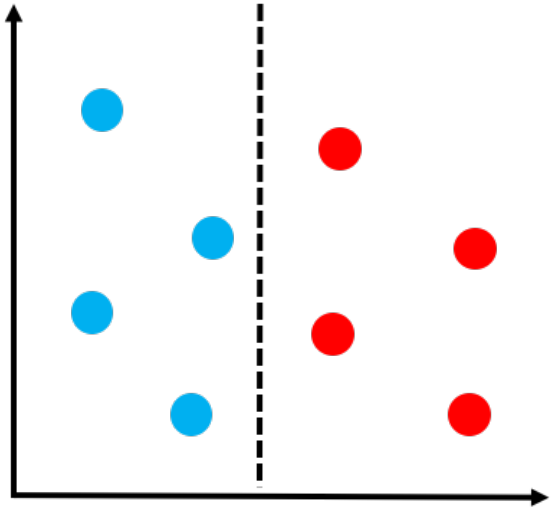
$$I_{left} = \frac{2}{2}\left(1 - \frac{2}{2}\right) + \frac{0}{2}\left(1 - \frac{0}{2}\right) = 0$$

$p = \frac{2}{6}, p = \frac{4}{6}$

$$I_{right} = \frac{2}{6}\left(1 - \frac{2}{6}\right) + \frac{4}{6}\left(1 - \frac{4}{6}\right) = \frac{4}{9}$$

$$\Delta I = \frac{1}{2} - 0\frac{2}{8} - \frac{4}{9}\frac{6}{8} = \frac{1}{6}$$

Best split



$p = \frac{4}{4}, p = \frac{0}{4}$

$$I_{left} = \frac{4}{4}\left(1 - \frac{4}{4}\right) + \frac{0}{4}\left(1 - \frac{0}{4}\right) = 0$$

$p = \frac{0}{4}, p = \frac{4}{4}$

$$I_{right} = \frac{0}{4}\left(1 - \frac{0}{4}\right) + \frac{4}{4}\left(1 - \frac{4}{4}\right) = 0$$

$$\Delta I = \frac{1}{2} - 0\frac{4}{8} - 0\frac{4}{8} = \frac{1}{2}$$

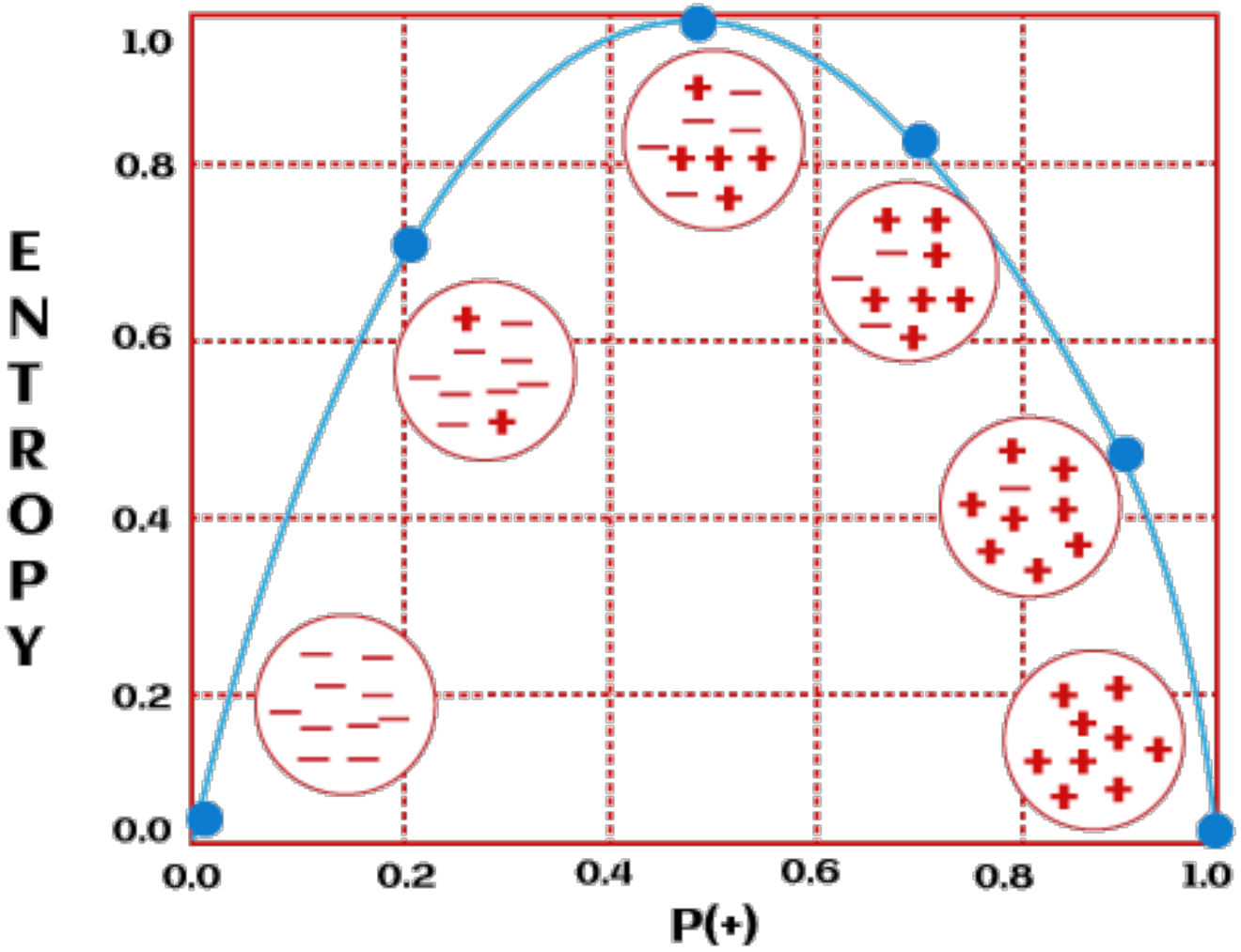
Энтропийный критерий (Entropy)

$$I_{node} = -\sum_{k=1}^{K} p_k \log p_k$$

$$p_k = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} [y_i = k]$$

- p_k–вероятность класса для вершины дерева
- K – общее число классов в задаче
- N_{node} – число объектов в вершине дерева

Энтропийный критерий (Entropy)



Источник: https://www.javatpoint.com/entropy-in-machine-learning

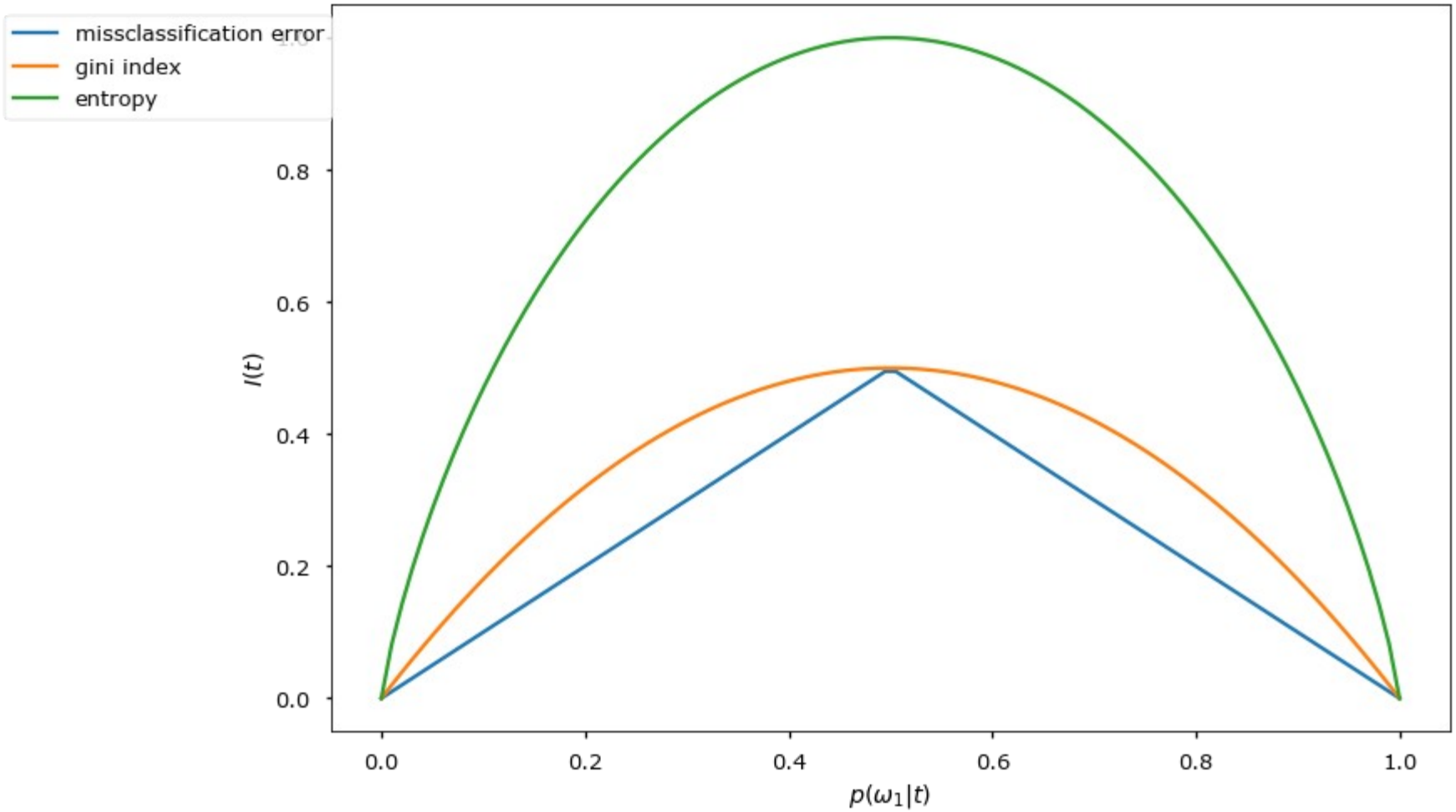
Ошибка классификации

$$I_{node} = 1 - \max_k \mathrm{p_k}$$

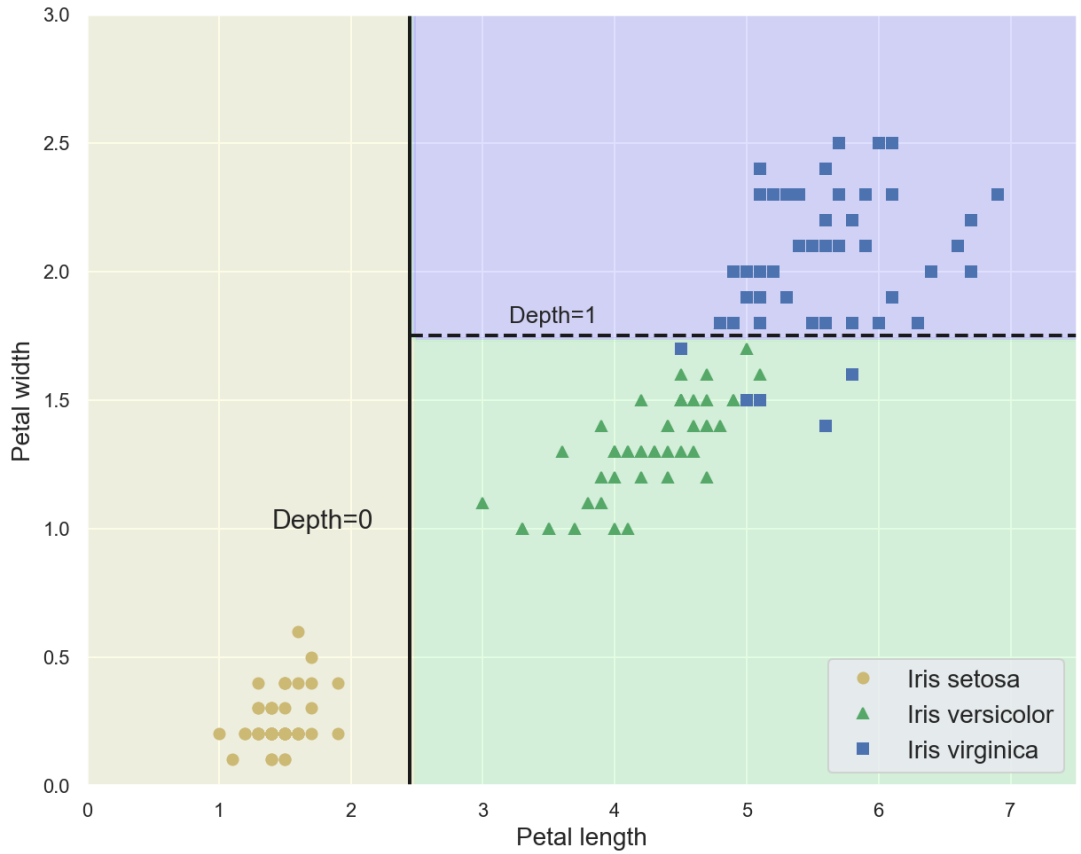
$$p_k = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} [y_i = k]$$

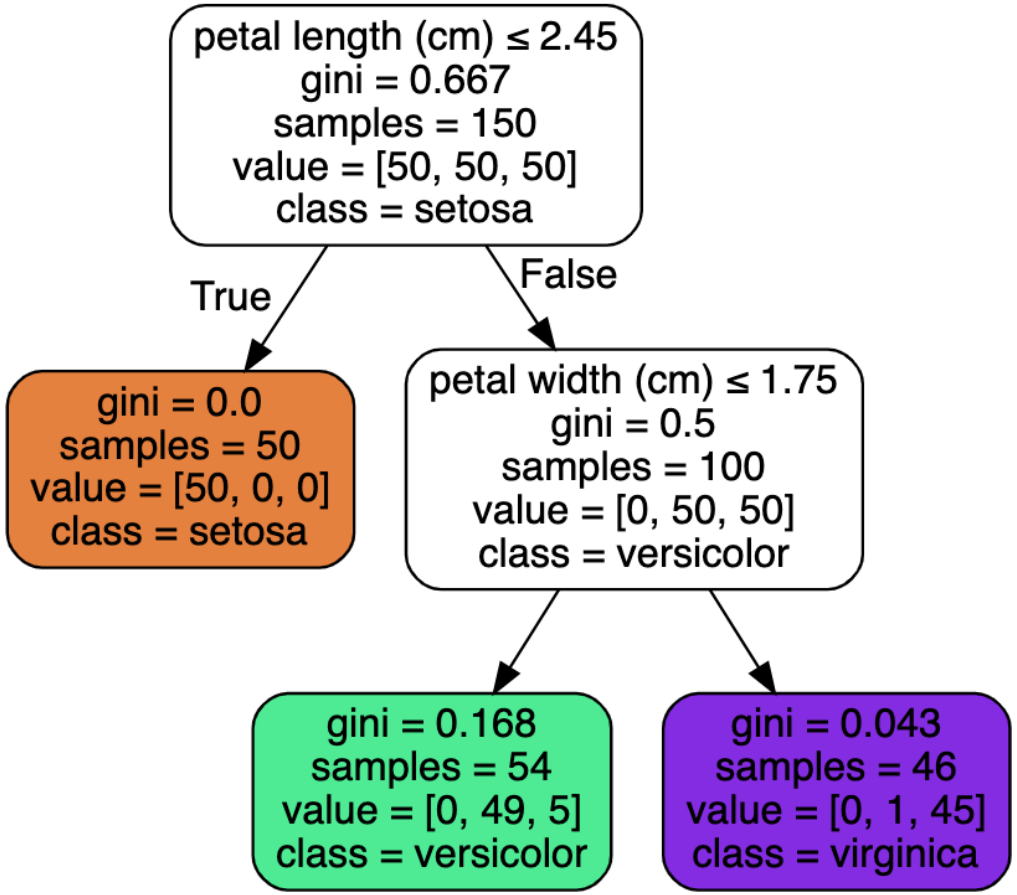
- p_k–вероятность класса для вершины дерева
- K – общее число классов в задаче
- N_{node} – число объектов в вершине дерева

Критерии


missclassification error
gini index
entropy
0.8
0.6
0.4
0.2
0.0
I(t)
0.0
0.2
0.4
0.6
0.8
1.0
p(ω1|t)

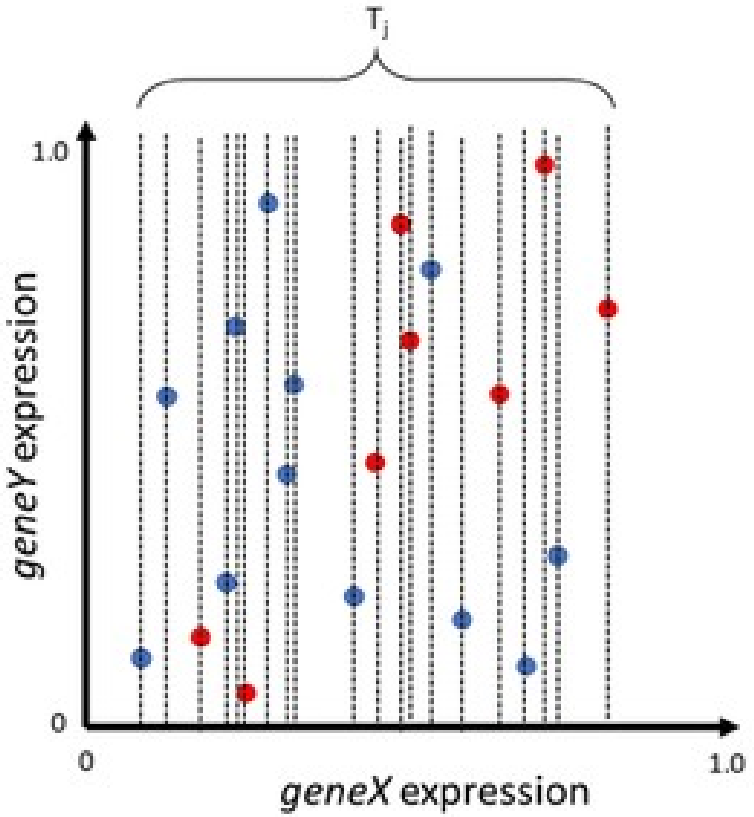
Пример

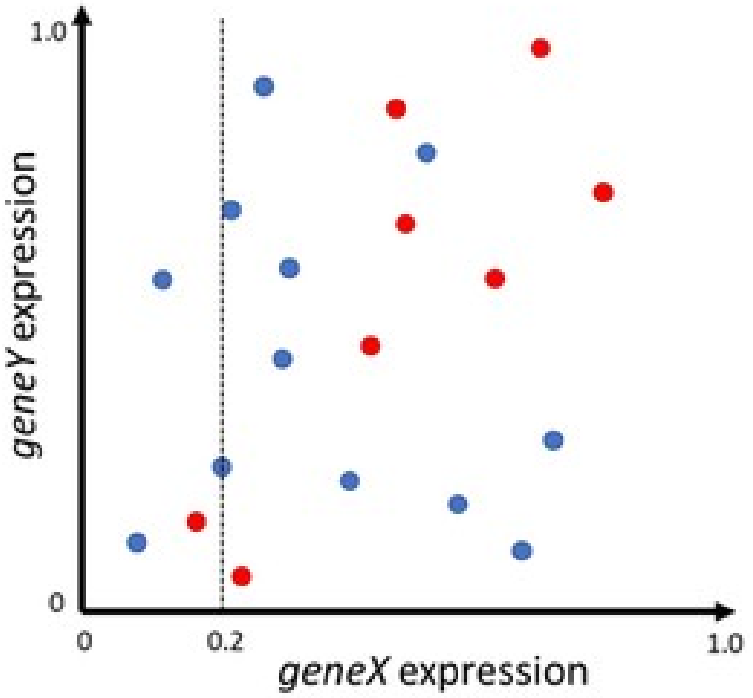

Depth=1
Depth=0
Petal width
Petal length
Iris setosa
Iris versicolor
Iris virginica

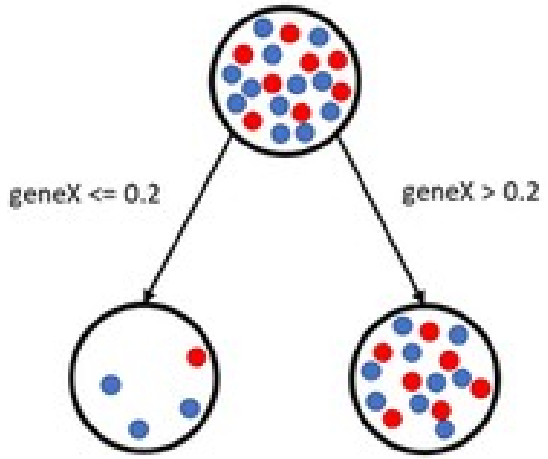

petal length (cm) ≤ 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
petal width (cm) ≤ 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica

Строим дерево. Шаг 1

i. For each feature, j, consider all possible thresholds T_j and calculate the cost function for each split





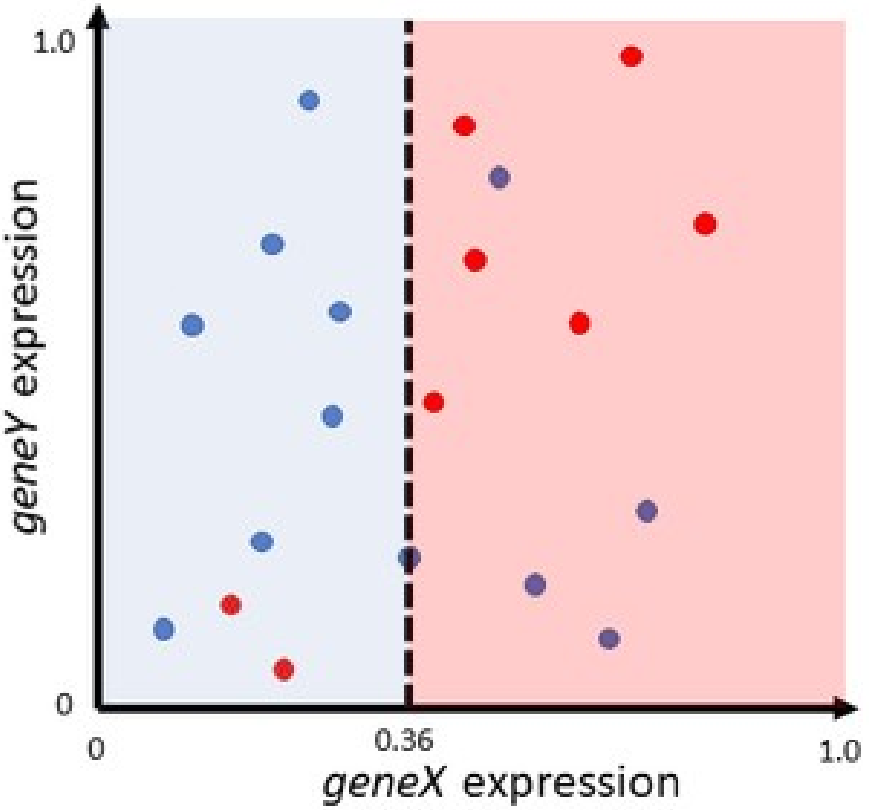


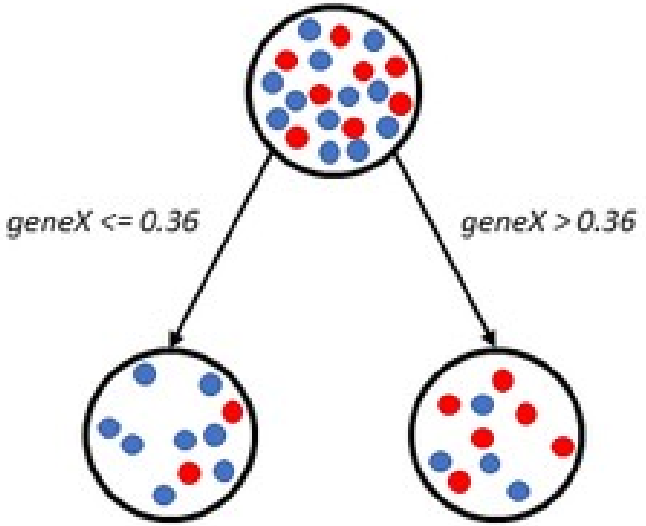
Information Gain = 0.017

Источник: https://www.researchgate.net/figure/Decision-tree-construction-protocol-with-the-Classification-and-Regression-tree-CART_fig2_347868061

Строим дерево. Шаг 2

ii. Identify the best feature and threshold pair (j*, t*), that returns the purest daughter nodes

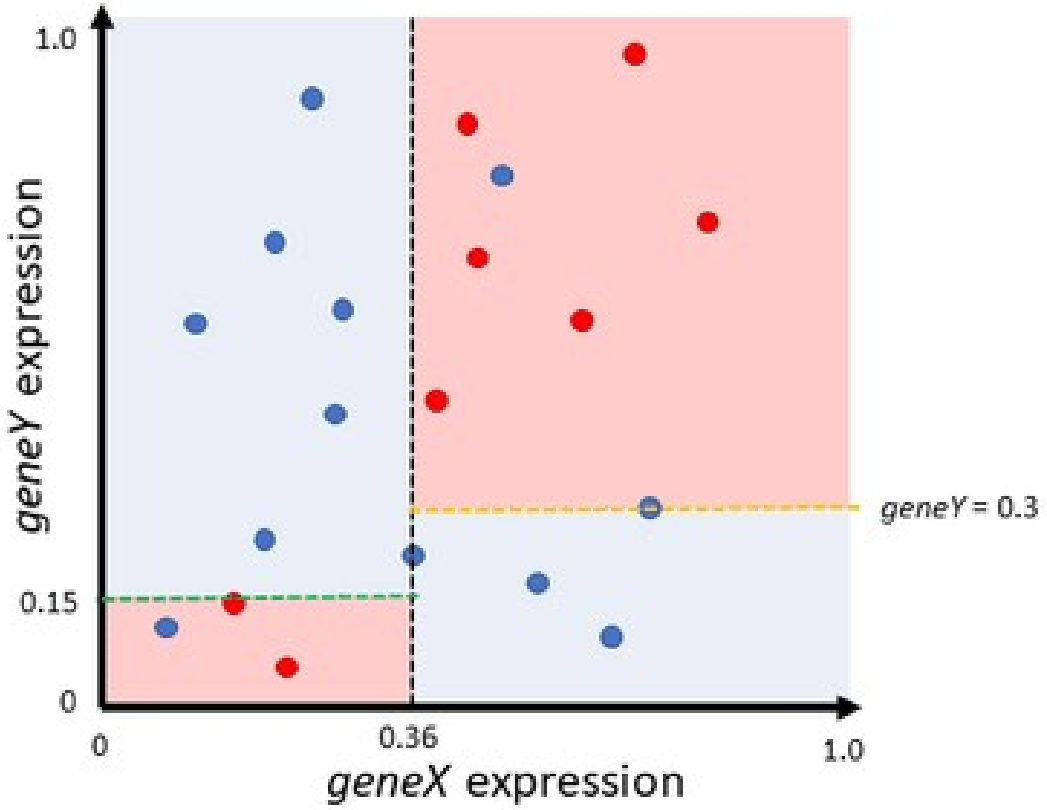


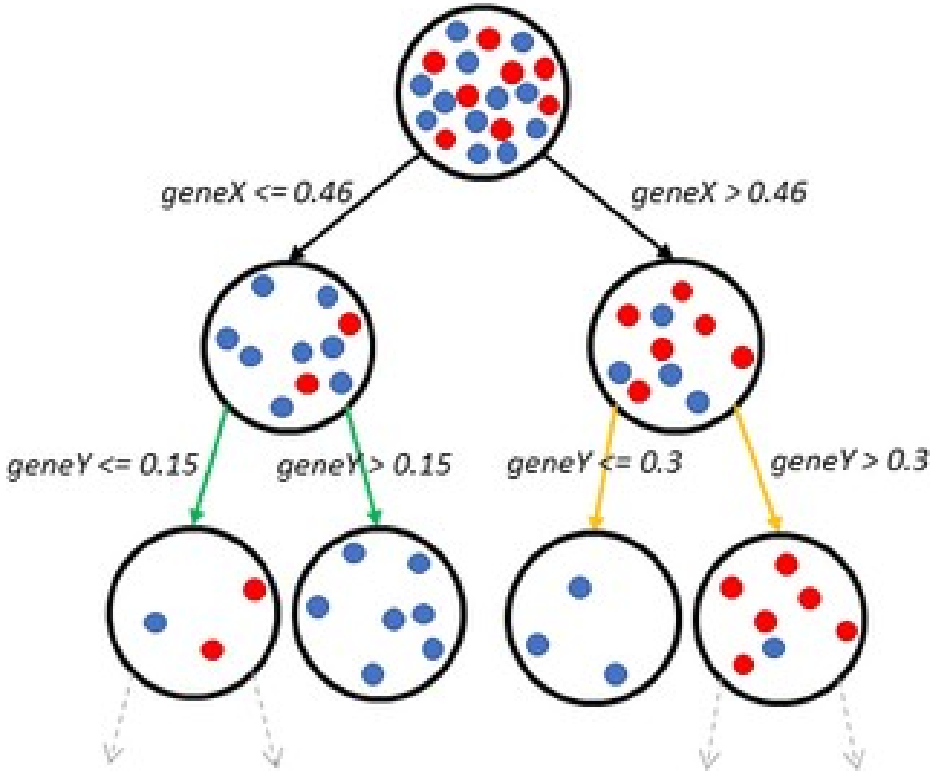


Information Gain = 0.124

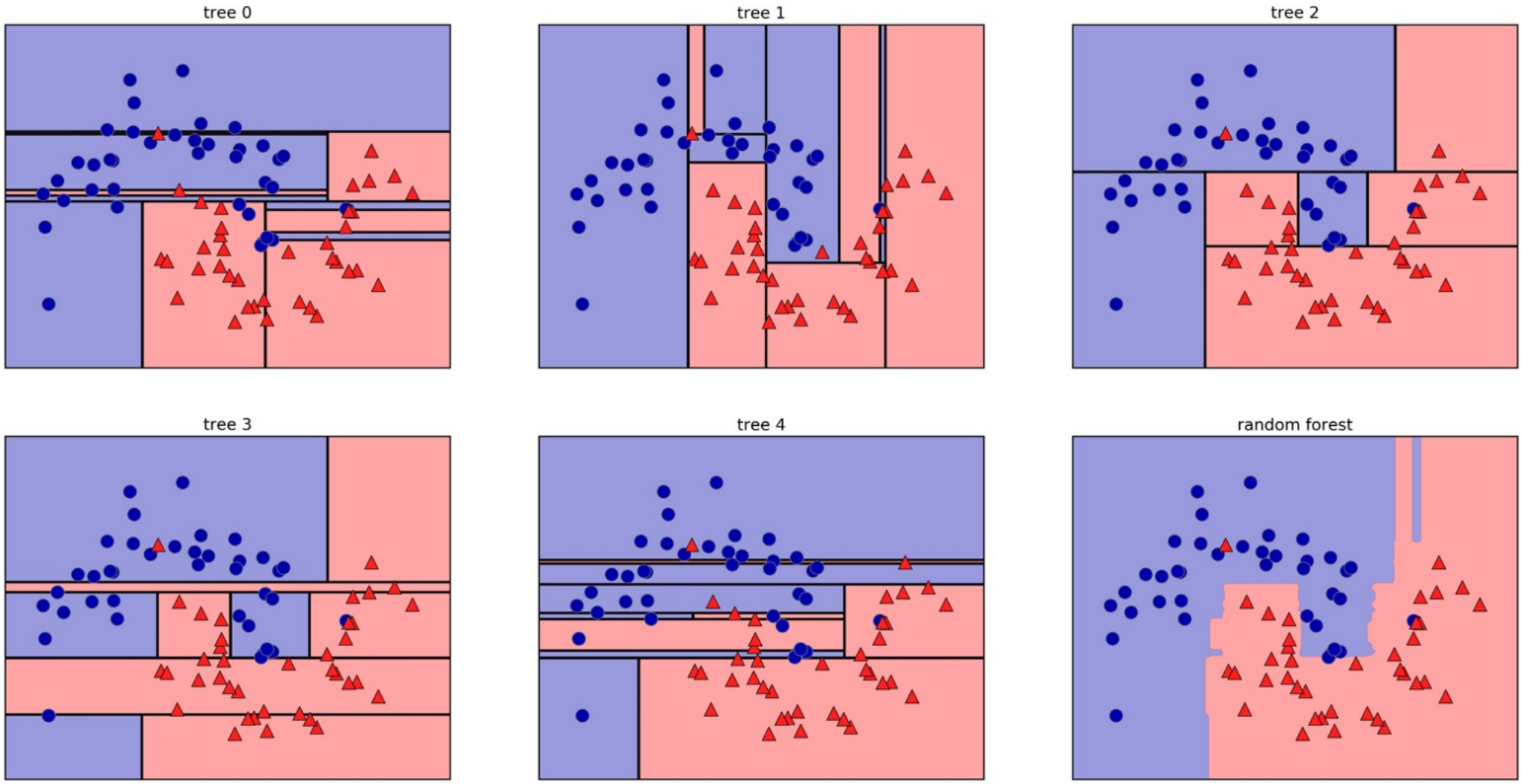
Строим дерево. Шаг 3

iii. Recurse on the daughter nodes to grow the tree, considering the next best feature-threshold pair





Пример



Критерии информативности для регрессии



Функционал качества предиката

- Для поиска лучшего предиката для вершины дерева зададим его функционал качества:

$$\Delta I_{node} = I_{node} - \left(I_{left} \frac{N_{left}}{N_{node}} + I_{right} \frac{N_{fight}}{N_{node}} \right) \rightarrow \max_{j,t}$$

  - I_{node} - **критерий информативности** для вершины
  - N_{node} - число объектов в левой вершине дерева
  - ΔI_{node} - функционал качества предиката (**information gain**)

Среднеквадратичная ошибка (MSE)

$$I_{node} = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} (y_i - \mu)^2$$

$$\mu = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} y_i$$

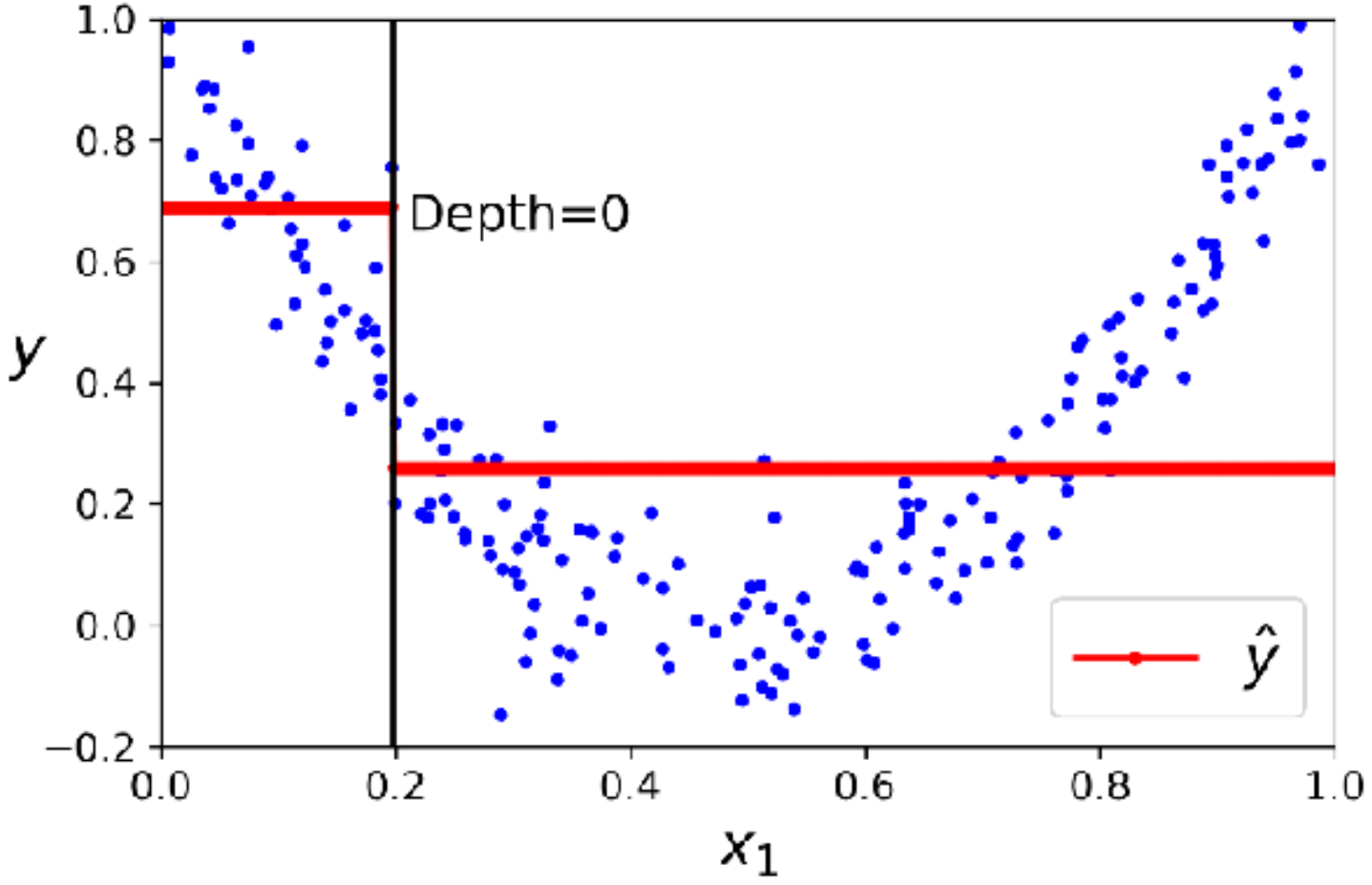
- μ – **среднее** значения целевой переменной в вершине дерева
- N_{node} – число объектов в вершине дерева

Средняя абсолютная ошибка (MAE)

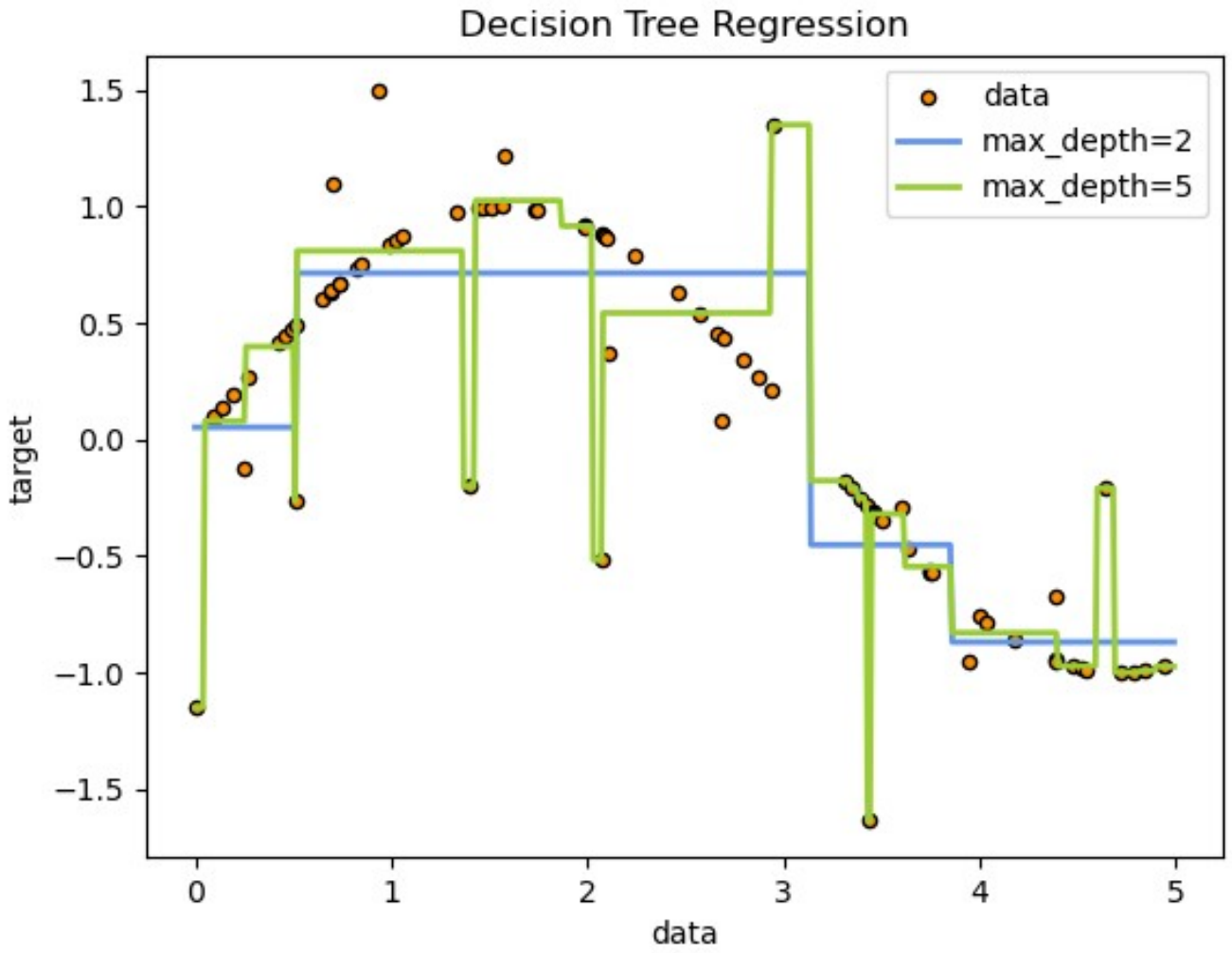
$$I_{node} = \frac{1}{N_{node}} \sum_{i=1}^{N_{node}} |y_i - \mu|$$

- μ – **медианное** значения целевой переменной в вершине дерева
- N_{node} – число объектов в вершине дерева

Пример



Пример


Decision Tree Regression
data
max_depth=2
max_depth=5
target
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0
1
2
3
4
5

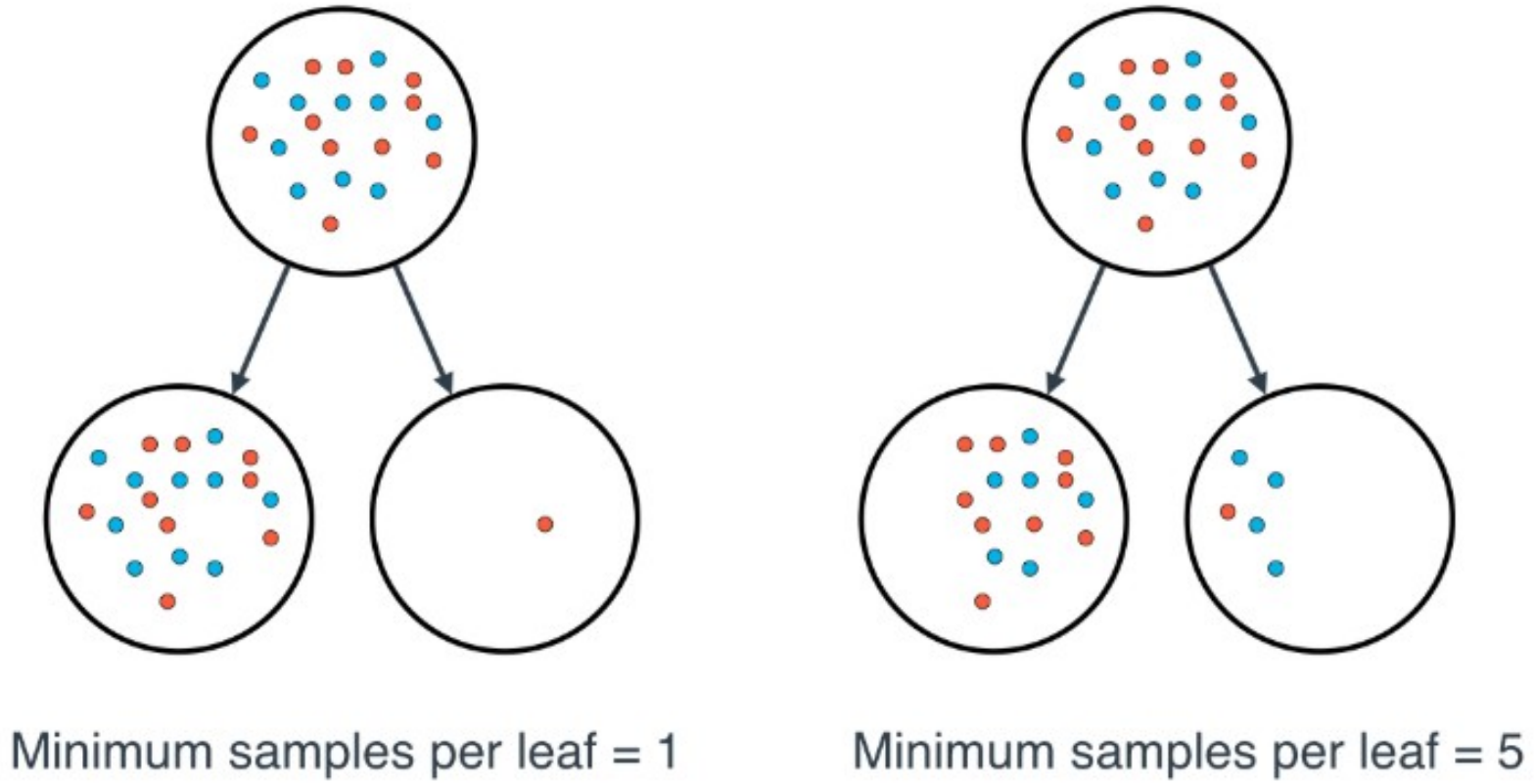
Критерии останова



Критерии останова

- Мы строим дерево до тех пор, пока не выполнится некоторые условия – критерии останова
- **Примеры критериев:**
  - Максимальная глубина дерева
  - Минимальное число объектов внутри листа дерева
  - Максимальное число листьев дерева
  - Останов, если все объекты в вершине относятся к одному классу
  - Требование, чтобы $\Delta I_{node} > h$

Минимальное число объектов внутри листа

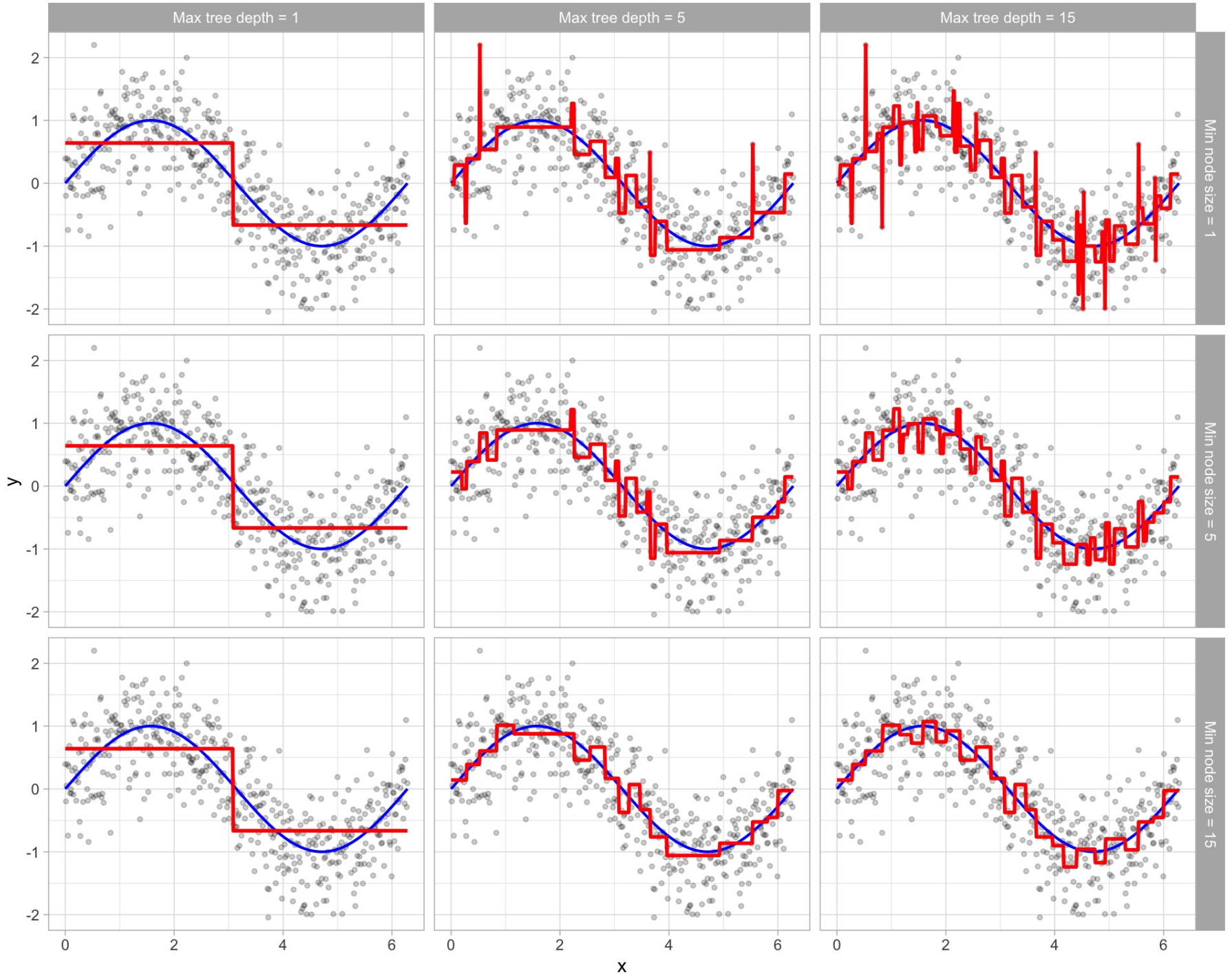


Minimum number of samples per leaf

Глубина дерева

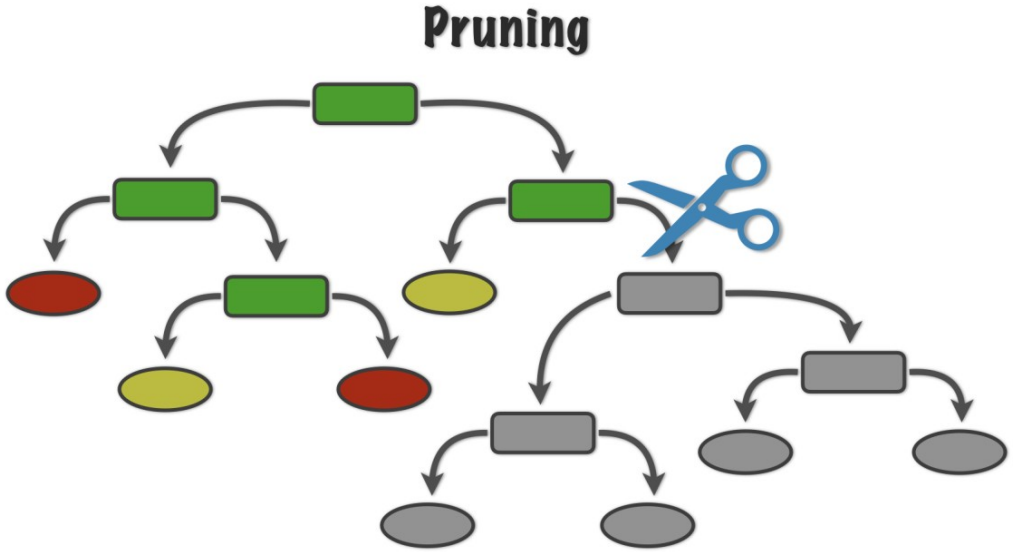
- Критерии останова влияют на качество прогнозов дерева
- Критерии останова влияют на глубину решающего дерева
- Очень глубокие деревья **переобучаются**
- Неглубокие деревья **недообучаются**

Пример

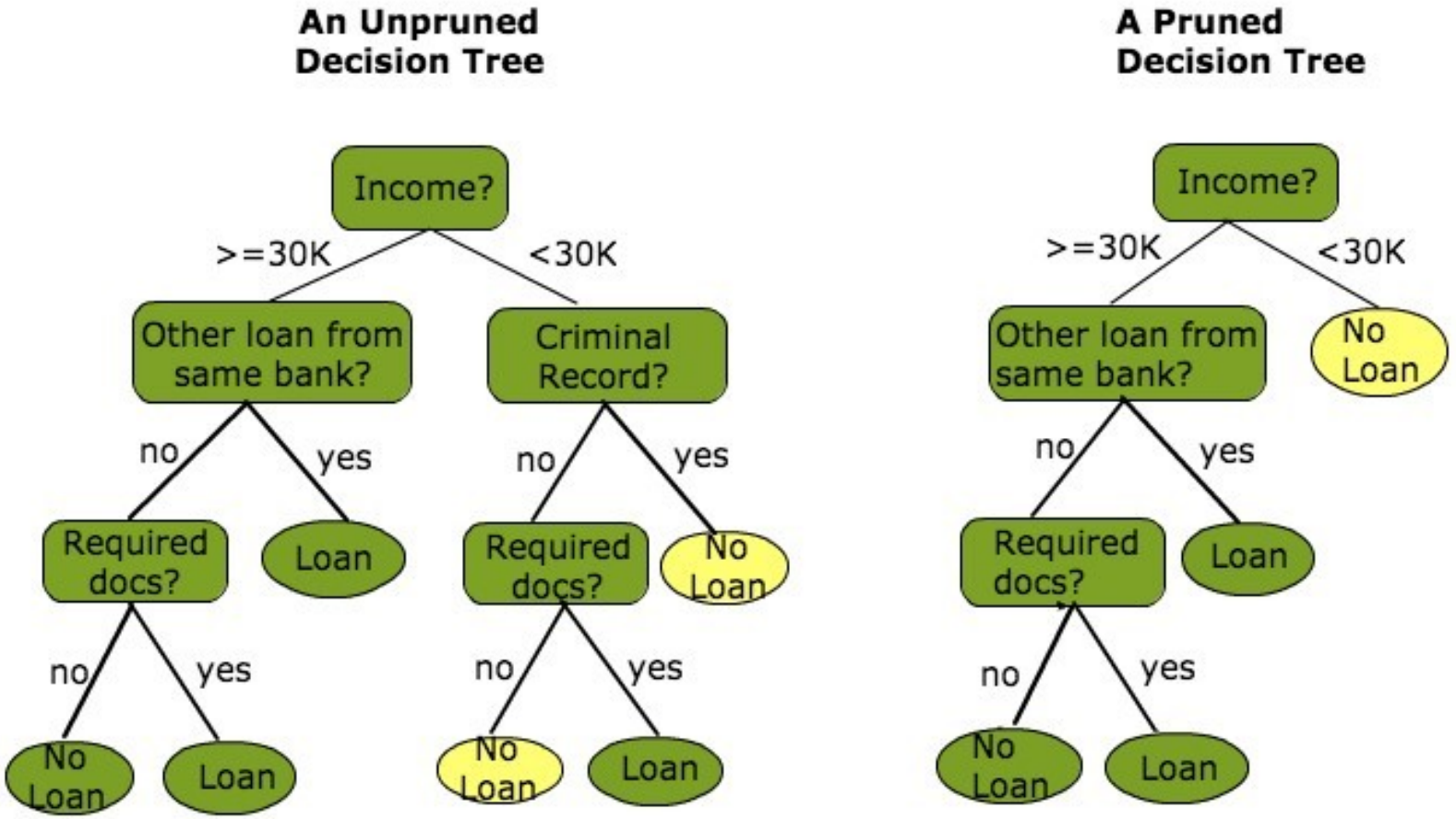

Max tree depth = 1
Max tree depth = 5
Max tree depth = 15
Min node size = 1
Min node size = 5
Min node size = 15
y
x
2
1
0
-1
-2
0
2
4
6

Стрижка (pruning) деревьев

- Строим переобученное дерево
- Отрезаем его «ветки» так, чтобы на тестовой выборке было наилучшее качество
- Есть несколько методов стрижки деревьев
- На практике применяется редко



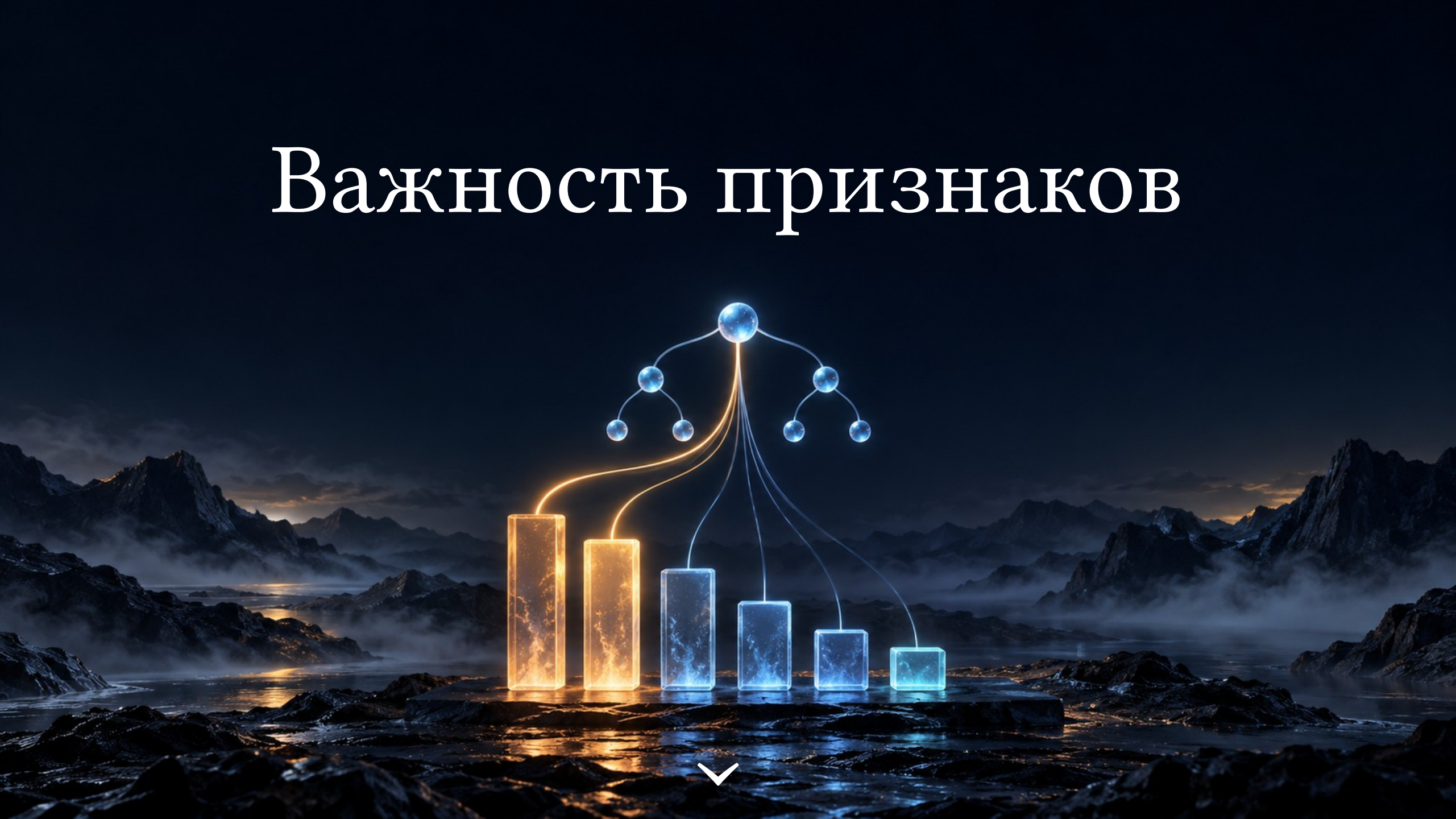
Пример


An Unpruned Decision Tree
Income?
>=30K
<30K
Other loan from same bank?
Criminal Record?
no
yes
no
yes
Required docs?
Loan
Required docs?
No Loan
no
yes
no
yes
No Loan
Loan
No Loan
Loan
A Pruned Decision Tree
Income?
>=30K
<30K
Other loan from same bank?
No Loan
no
yes
Required docs?
Loan
no
yes
No Loan
Loan

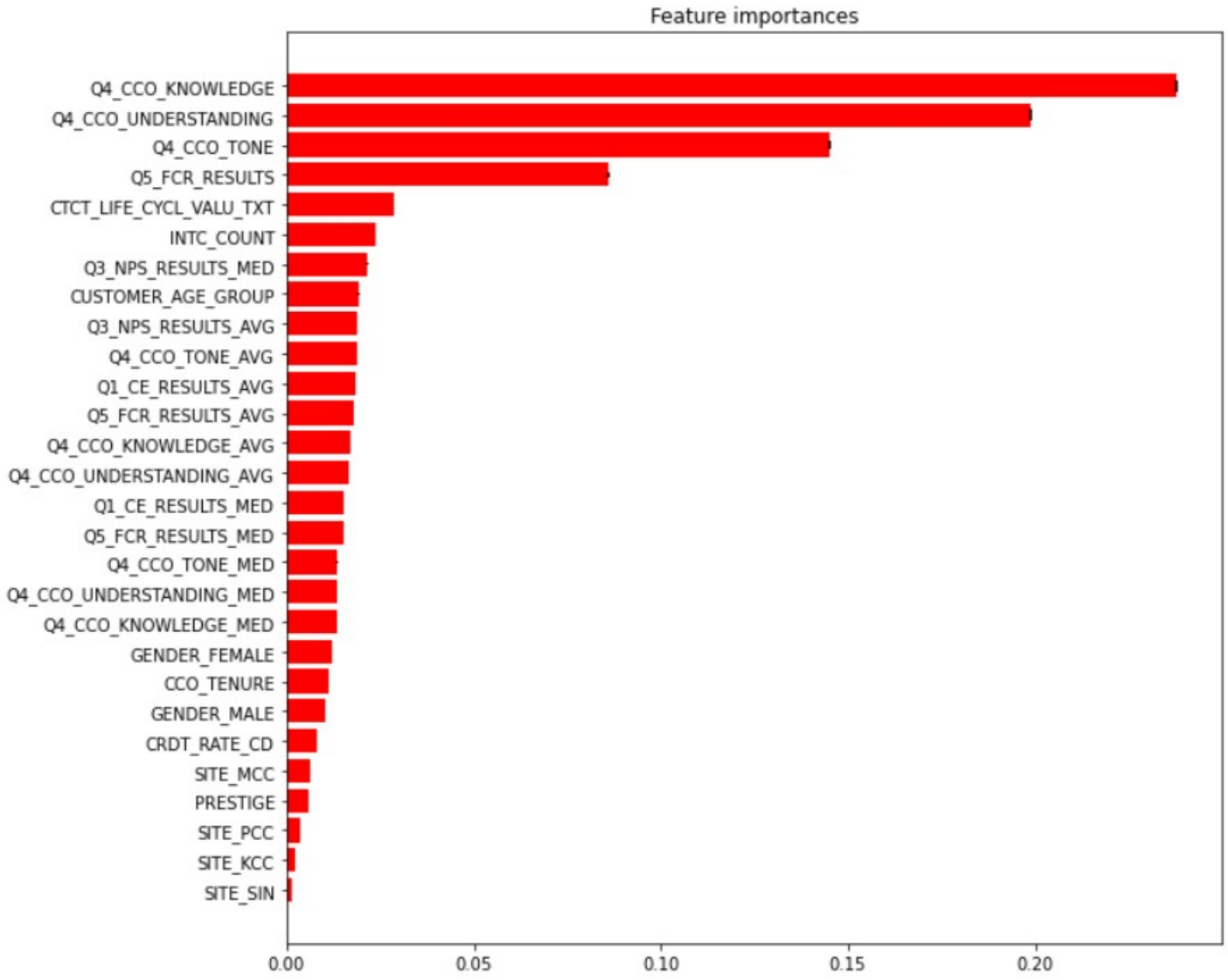
Пример


1. До обрезки
Возраст ≤ 30
Да
Нет
Доход ≤ 50
Купит
Да
Нет
Не купит
Визитов ≤ 3
Да
Нет
Не купит
Купит
Сложная ветвь:
может подстраиваться
под шум
2. Заменяем ветвь листом
Весь подграф заменяется одним листом.
Визитов ≤ 3
Да
Нет
Не купит
Купит
Не купит
Для классификации лист предсказывает
самый частый класс среди объектов этой ветви.
Обрезаем, если упрощение
оправдывает потерю точности
на обучении.
Пример: cost-complexity pruning
Критерий = ошибка на обучении + α × число листьев
Больше α → сильнее обрезка.
3. После обрезки
Один шаг обрезки
Возраст ≤ 30
Да
Нет
Доход ≤ 50
Купит
Да
Нет
Не купит
Не купит
Новый лист
На обучении точность может снизиться.
На новых данных качество может улучшиться.
Предварительная остановка
Ограничиваем рост: max_depth, min_samples_leaf.
Постобрезка
Сначала выращиваем дерево, затем удаляем ветви.
Силу обрезки выбирают по валидации или кросс-валидации.

Важность признаков



Важность признаков



Важность признаков

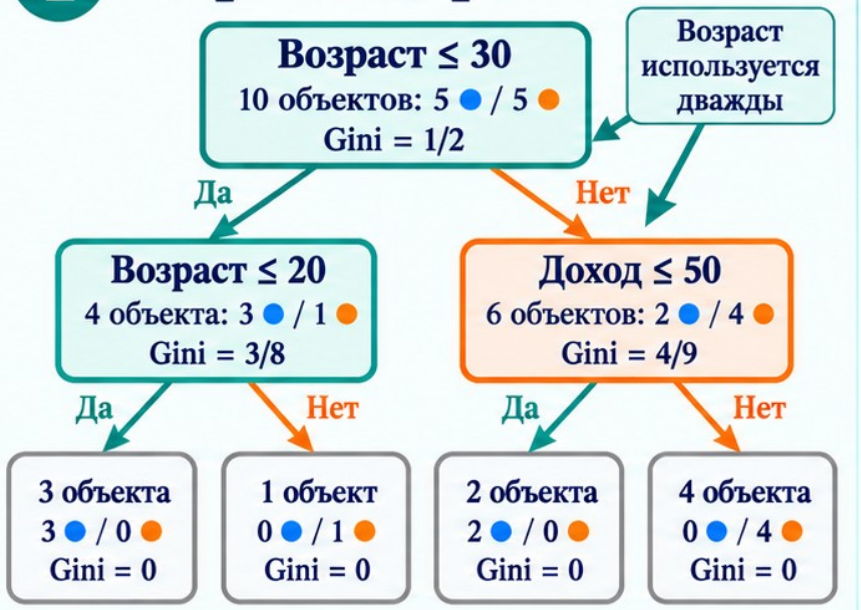
- Решающее дерево позволяет оценить вклад каждого входного признака в прогноз
- **Важность признака *j*** (feature importance):

$$FIM_j = \sum_{node \in T(j)} N_{node} \Delta I_{node}$$

– $T(j)$ – **все вершины дерева, где** предикаты использовали признак ***j***: $\{x_j > t\}$

Пример

1 Строим дерево



2 Считаем вклад вершин

$$\Delta = \frac{n}{N} \times \left[\text{Gini} - \frac{n_L}{n} \times \text{Gini}_L - \frac{n_R}{n} \times \text{Gini}_R \right]$$

N — все объекты; n — объекты в вершине.

Возраст: корневая вершина

$$\Delta_1 = 1 \times \left[\frac{1}{2} - \frac{4}{10} \times \frac{3}{8} - \frac{6}{10} \times \frac{4}{9} \right] = \frac{1}{12}$$

Возраст: левая вершина

$$\Delta_2 = \frac{4}{10} \times \left[\frac{3}{8} - 0 - 0 \right] = \frac{3}{20}$$

Доход: правая вершина

$$\Delta_3 = \frac{6}{10} \times \left[\frac{4}{9} - 0 - 0 \right] = \frac{4}{15}$$

3 Складываем и нормируем

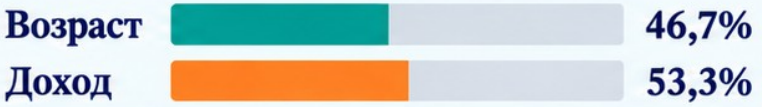
Вклад возраста

$$\Delta_1 + \Delta_2 = \frac{1}{12} + \frac{3}{20} = \frac{7}{30}$$

Вклад дохода

$$\Delta_3 = \frac{4}{15}$$

Общий вклад: $\frac{7}{30} + \frac{4}{15} = \frac{1}{2}$



Важность = вклад признака / общий вклад.

Если признак встречается в нескольких вершинах, его вклады суммируются.

Метод MDI: взвешенное уменьшение неоднородности. Gini = $1 - \sum p_k^2$.

Обработка пропусков

- **Обучение** решающего дерева:
  - Пусть для признака j некоторые значения пропущены
  - При поиске лучшего предиката $\{x_j > t\}$ не используем пропуски x_j. Считаем ΔI_{node} только на известных x_j.

- **Применение** решающего дерева:
  - Если пропущено значение $x_j = None$ для объекта, то идем и в лево, и в право. Делаем прогноз в обоих поддеревьях
  - Итоговый прогноз $\hat{y}$ для объекта усредняем:

$$\hat{y} = \frac{N_{left}}{N_{node}} \hat{y}_l + \frac{N_{right}}{N_{node}} \hat{y}_r$$

Методы построения деревьев

- ID3
  - Энтропийный критерий
  - Только категориальные признаки
- C4.5
  - Критерий Gain Ratio
  - Error-Based Prunning
  - Обработка пропусков как описали выше
- CART (sklearn)
  - Критерий Джини
  - Cost-Complexity Pruning
  - Метод суррогатный предикатов для обработки пропусков

Заключение



Резюме

- Даны данные X, y
- Находим **предикат** $\{x_j > t\}$ для вершины дерева:

$$\Delta I_{node} = I_{node} - \left(I_{left} \frac{N_{left}}{N_{node}} + I_{right} \frac{N_{fight}}{N_{node}} \right) \to \max_{j,t}$$

- Строим две **дочерние вершины**: правую и левую

$$R_r(j,t) = \{x | x_j > t\}$$

$$R_l(j,t) = \{x | x_j \leq t\}$$

- **Рекурсивно** находим предикаты для всех дочерних вершин
- Если выполняются **критерии останова**, останавливаемся

Демки

- https://mlu-explain.github.io/decision-tree/
- http://www.r2d3.us/visual-intro-to-machine-learning-part-1/

Вопросы

- Что такое решающее дерево? Запишите формулу предсказания решающего дерева (через разбиение признакового пространства на области).
- Опишите жадный алгоритм обучения решающего дерева. Запишите функционал, который оптимизируется при построении вершины решающего дерева.
- Какими основными свойствами должен обладать критерий информативности? Как он используется для выбора предиката во внутренней вершине решающего дерева?
- Запишите критерий Джини и энтропийный критерий информативности.